

Verilog In One Day

<https://www.asic-world.com/verilog/veritut.html>

A D-flip flop appears

```
module dff_with_inverter (  
    input wire clk,  
    input wire rst_n,  
    input wire d,  
    output wire q,  
    output wire q_bar  
);  
  
    reg q_reg;  
  
    // ---- Sequential logic ----  
    // Output depends on current input AND past state (memory)  
    // Only updates on the clock's rising edge  
    always @(posedge clk) begin  
        if (!rst_n)  
            q_reg <= 1'b0;  
        else  
            q_reg <= d;  
        end  
  
    // ---- Combinational logic ----  
    // Output depends only on current input, no memory, no clock  
    assign q      = q_reg;  
    assign q_bar = ~q_reg;  
  
endmodule
```

Basic Verilog Syntax

- Module definition syntax

```
module priority_decoder_4to2 (  
    input wire [3:0] in,          // 4 input request lines  
    output reg [1:0] code,        // encoded index of highest-priority input  
    output reg          valid     // 1 if any input is asserted  
> ); ...  
  
endmodule
```

- `input/output` are keywords to define direction of port (`inout` port can be used for both)
- `// ...` and `/* */` are for comments
- `wire/reg` define data types and `[N:0]` defines size; we will get to them later
- Each module definition must define ports (inputs & outputs) in parentheses
- Module definition ends with `endmodule`
- **Note:** Curly braces are replaced with explicit `begin, end...`-style statements in Verilog

Calling a Module

- define variables for **input/output**
for eg: `req_lines`, `priority_code`,
`req_valid`
- instantiate it with a **unique instance name**
for eg: `pdec_inst` is the unique identifier for this
instance of `priority_decoder_4to2`
- wire in the inputs/outputs using **port names**
for eg: `.in(req_lines)` wires in `req_lines` to the `in` port

```
reg [3:0] req_lines;    // input - reg
wire [1:0] priority_code; // output - wire
wire      req_valid;    // output - wire

priority_decoder_4to2 pdec_inst (
    .in      (req_lines),
    .code    (priority_code),
    .valid    (req_valid)
);
```

And two roads diverged...

- You will think of digital logic as either combinational or sequential
- **Combinational** — output depends only on current inputs; no memory, no clock

```
// ---- Combinational logic ----  
// Output depends only on current input  
assign q      = q_reg;  
assign q_bar = ~q_reg;
```

- **Sequential** — output depends on current inputs and past state; needs a clock to advance

```
// ---- Sequential logic ----  
// Output depends on current input  
// AND past state (memory)  
// Only updates on the clock's rising edge  
always @(posedge clk) begin  
    if (!rst_n)  
        q_reg <= 1'b0;  
    else  
        q_reg <= d;  
end
```

Data Types

Verilog supports two data types for these purposes:

- **wire** – carries a value **continuously** from whatever drives it; has no memory of its own
 - **assign** is used for **continuous assignment** to a wire. It cannot be used for register data.
 - Eg: `assign q_bar = ~q_reg;`
- **reg** – a variable that **holds its value** until explicitly assigned inside a procedural block
 - **Note:** The definition is **logical**. It is not guaranteed to synthesize to a **physical register**
 - Can only be assigned inside of `always @(...)` blocks
 - Usage: `always @(trigger-list)`
 - Eg: `always @(*)` executes whenever **any input changes** -> **combinational**
 - Eg: `always @(posedge clk)` executes on **rising edge of clk** signal -> **sequential**

Two ways to do assignment

- Inside **always** blocks we can write **combinational** (using **@(*)**) or **sequential** (eg. using **@(posedge clk)**) logic

Combinational

- requires **no physical registers**
- **ALL updates are executed at once**
- (typically) blocking assignment (**=**) is used

```
always @(*) begin
    c = a & b;
    d = a | b;
    out1 = d;
    out2 = c ^ b;
end
```

Sequential

- (usually) requires **physical registers**
- updates transferred **sequentially ONE-by-ONE**
- (typically) non-blocking assignment (**<=**) is used

```
always @(posedge clk) begin
    if (!rst_n)
        q_reg <= 1'b0;
    else
        q_reg <= d;
end
```

Back to Basics

Most C-style **operators** are supported:

- Bitwise: `& | ^ ~` — per-bit operations
- Logical: `&& || !` — collapse to true/false, used in conditions
- Comparison: `== != < >`
- Concatenation: `{a, b}` — combines signals into a wider signal vector

Control constructs (allowed inside `always` / `initial` only)

- `if / else` — same as software
- `case` — switch-statement for hardware

Back to Basics

Loop constructs: Can appear either as **logical loops** (in testbenches) or **repeated hardware** (in synthesizable RTL).

- `for(init; cond; update)` loop – used in testbenches OR for repeating a hardware unit
- `repeat(N)` loop – used in testbenches OR for repeating a hardware unit
- `while(cond)` loop – used in testbenches

Example: *(repeating hardware)*

```
repeat(4) begin
|   c[i] = a[i] ^ b[i];
end
```

(writing testbench code)

```
for (i = 0; i < 32; i = i + 1)
|   $display("c[i]=%b", c[i])
```

Back to Basics

Procedural blocks: All assignments (blocking `=` or non-blocking `<=`) occur within procedural blocks.

- `initial` block – used in testbenches for doing setup work at T=0
- `always @(trigger-list)` block – runs forever! You will use this often for comb/seq logic

Example: *(setup in testbench)*

```
initial begin
    $monitor("t=%0t rst_n=%b d=%b q=%b q_bar=%b",
             $time, rst_n, d, q, q_bar);
end
```

(defining combinational logic)

```
always @(*) begin
    // defaults -- avoids accidental latch inference
    code = 2'b00;
    valid = 1'b1;

    casez (in)
        4'b1???: code = 2'b11;    // in[3] has highest priority
        4'b01??: code = 2'b10;    // in[2]
        4'b001?: code = 2'b01;    // in[1]
        4'b0001: code = 2'b00;    // in[0] has lowest priority
        default: begin
            code = 2'b00;
            valid = 1'b0;          // no input asserted
        end
    endcase
end
```

Example

```
always @(*) begin
    // defaults -- avoids accidental latch inference
    code = 2'b00;
    valid = 1'b1;

    casez (in)
        4'b1???: code = 2'b11;    // in[3] has highest priority
        4'b01??: code = 2'b10;    // in[2]
        4'b001?: code = 2'b01;    // in[1]
        4'b0001: code = 2'b00;    // in[0] has lowest priority
        default: begin
            code = 2'b00;
            valid = 1'b0;          // no input asserted
        end
    endcase
end
```

Example

```
always @(*) begin
    // defaults -- avoids accidental latch inference
    code = 2'b00;
    valid = 1'b1;

    casez (in)
        4'b1???: code = 2'b11;    // in[3] has highest priority
        4'b01??: code = 2'b10;    // in[2]
        4'b001?: code = 2'b01;    // in[1]
        4'b0001: code = 2'b00;    // in[0] has lowest priority
        default: begin
            code = 2'b00;
            valid = 1'b0;          // no input asserted
        end
    endcase
end
```

- If any case is left out (in **case** / **if-else** blocks), a latch would be inferred
- Use **begin** and **end** to mark multiline blocks

Example

```
always @(*) begin
    // defaults -- avoids accidental latch inference
    code = 2'b00;
    valid = 1'b1;

    casez (in)
        4'b1??? : code = 2'b11; // in[3] has highest priority
        4'b01?? : code = 2'b10; // in[2]
        4'b001? : code = 2'b01; // in[1]
        4'b0001 : code = 2'b00; // in[0] has lowest priority
        default: begin
            code = 2'b00;
            valid = 1'b0;
        end
    endcase
end
```

- Use **casez** for wildcard based pattern matching – order matters
- Use **?** for wildcard bit (0 or 1)

One isⁿt gud genug

Using vector signals in verilog is easy:

- **Declaring:** `wire/reg [MSB:LSB] var_name`

for eg: `wire [3:0] data` declares a wire that can carry 4 bits

- **Literals:** `<width>'<base><value>`

for eg: `4'b1010` (binary), `8'hA5` (hex), `4'd9` (decimal)

- **Assignment:**

`data = 4'b0110` (full assignment)

`data[1:0] = 2'b10` (multi-bit assignment)

`data[3:0] = {2'b01, 2'b10}` (using concat operator)

> this will assign the bits 3, 2, 1, 0 (left to right) to the values 0, 1, 1, 0.

- **Reading:** Same syntax as assignment

for eg: `data[high:low]` will read bits `low, low+1, ... high-1, high`

- Note: You can use the `[LSB:MSB]` convention as well in Verilog – stay consistent with one convention.

More data types

Verilog provides helper data types to make life easier – these do not map to real hardware (`wire/reg` do).

- **integer** – plain whole number variable, mostly used to count things (like loop iterations)
for eg: `integer i; for (i = 0; i < 16; i = i + 1) .`
- **real** – a decimal/floating-point number; **only used in simulation/testing**
for eg: `real voltage = 3.3 .`
- **time** – stores a **simulation/testing timestamp**, usually paired with `$time`
for eg: `time start_time; start_time = $time; .`
- **parameter** – a named constant you can **reconfigure** from outside the module
for eg: `parameter WIDTH = 8` – can write variable-width modules using this
- **localparam** – a named constant that is **fixed inside a module** – used to store helper values
for eg: `localparam OP_ADD = 4'b0000; .`

Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \
cin = %b -> cout = %b, sum = %b",
        $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
    end
endmodule
```

```
initial begin
    $dumpfile("task1.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; cin = 0; #10;
    a = 0; b = 0; cin = 1; #10;
    a = 0; b = 1; cin = 0; #10;
    a = 0; b = 1; cin = 1; #10;
    a = 1; b = 0; cin = 0; #10;
    a = 1; b = 0; cin = 1; #10;
    a = 1; b = 1; cin = 0; #10;
    a = 1; b = 1; cin = 1; #10;

    $display("Done!")
    $finish;
end
endmodule
```


Testing Verilog

```
`timescale 1ns/1ns

module tb;
    reg a, b, cin;
    wire cout, sum;

    adder uut (
        .a(a),
        .b(b),
        .cin(cin),
        .cout(cout),
        .sum(sum)
    );

    initial begin
        $monitor("At time %0t: a = %b, b = %b, \n",
            cin = %b -> cout = %b, sum = %b",
            $time, a, b, cin, cout, sum);
    end

    initial begin
        $dumpfile("task1.vcd");
        $dumpvars(0, tb);
    end
endmodule
```

> A testbench for task1

- ``timescale <unit>/<precision>` for simulation
- N time units are represented as `#N`.
- eg: `#5` represents a delay of 5ns here
- use `reg` to store inputs, and `wire` to take outputs
- `initial` block will run once at T=0; used for setup code

Testing Verilog

```
`timescale 1ns/1ns
```

```
module tb;
```

```
  reg a, b, cin;
```

```
  wire cout, sum;
```

```
  adder uut (
```

```
    .a(a),
```

```
    .b(b),
```

```
    .cin(cin),
```

```
    .cout(cout),
```

```
    .sum(sum)
```

```
);
```

```
initial begin
```

```
  $monitor("At time %0t: a = %b, b = %b, \\  
    cin = %b -> cout = %b, sum = %b",  
    $time, a, b, cin, cout, sum);
```

```
end
```

```
initial begin
```

```
  $dumpfile("task1.vcd");
```

```
  $dumpvars(0, tb);
```

- we define a module to define and run the testbench
- instantiate the module to be tested and drive inputs, define outputs
- `$monitor` prints a message anytime one of the **inputs** changes

Testing Verilog

```
initial begin
    $dumpfile("task1.vcd");
    $dumpvars(0, tb);

    a = 0; b = 0; cin = 0; #10;
    a = 0; b = 0; cin = 1; #10;
    a = 0; b = 1; cin = 0; #10;
    a = 0; b = 1; cin = 1; #10;
    a = 1; b = 0; cin = 0; #10;
    a = 1; b = 0; cin = 1; #10;
    a = 1; b = 1; cin = 0; #10;
    a = 1; b = 1; cin = 1; #10;

    $display("Done!");
    $finish;
end
endmodule
```

- `$dumpfile` and `$dumpvars` tell the simulator where to store waveform data and which variables to dump
- define delays using `#10` etc.
- `$display` prints a message once
- `$finish` tells the simulator to end simulating

Testing Verilog

Output from `vvp` command:

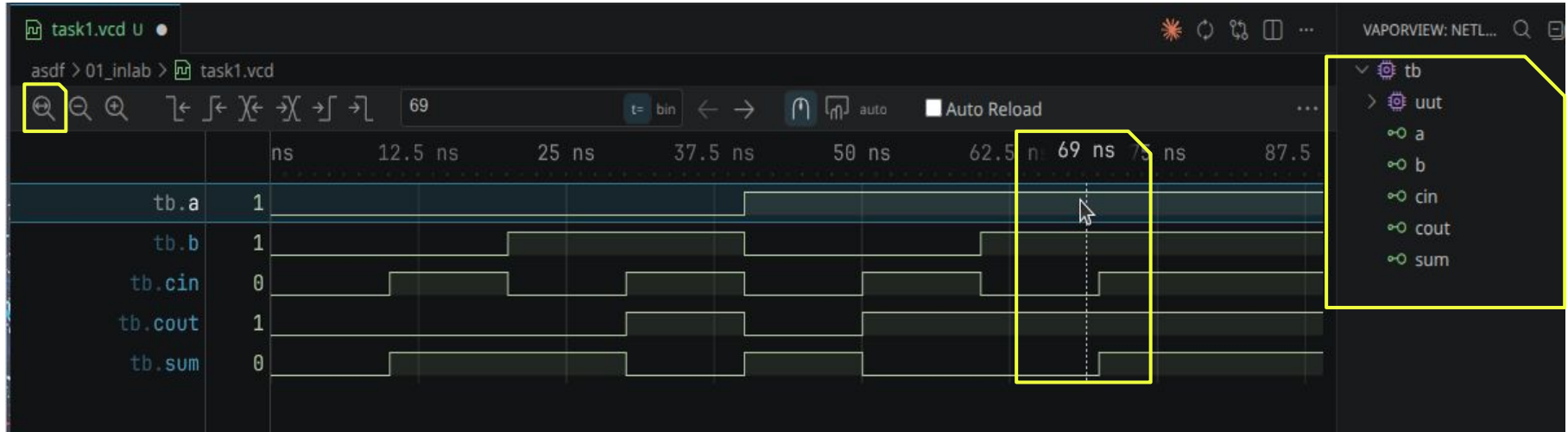
```
VCD info: dumpfile task1.vcd opened for output.  
At time 0: a = 0, b = 0,      cin = 0 -> cout = 0, sum = 0  
At time 10: a = 0, b = 0,     cin = 1 -> cout = 0, sum = 1  
At time 20: a = 0, b = 1,     cin = 0 -> cout = 0, sum = 1  
At time 30: a = 0, b = 1,     cin = 1 -> cout = 1, sum = 0  
At time 40: a = 1, b = 0,     cin = 0 -> cout = 0, sum = 1  
At time 50: a = 1, b = 0,     cin = 1 -> cout = 1, sum = 0  
At time 60: a = 1, b = 1,     cin = 0 -> cout = 1, sum = 0  
At time 70: a = 1, b = 1,     cin = 1 -> cout = 1, sum = 1  
Done!  
task1/tb_task1.v:35: $finish called at 80 (1ns)
```

Compiling Verilog

```
$ iverilog adder.v tb_task1.v -o task1.vvp  
$ vvp task1.vvp
```

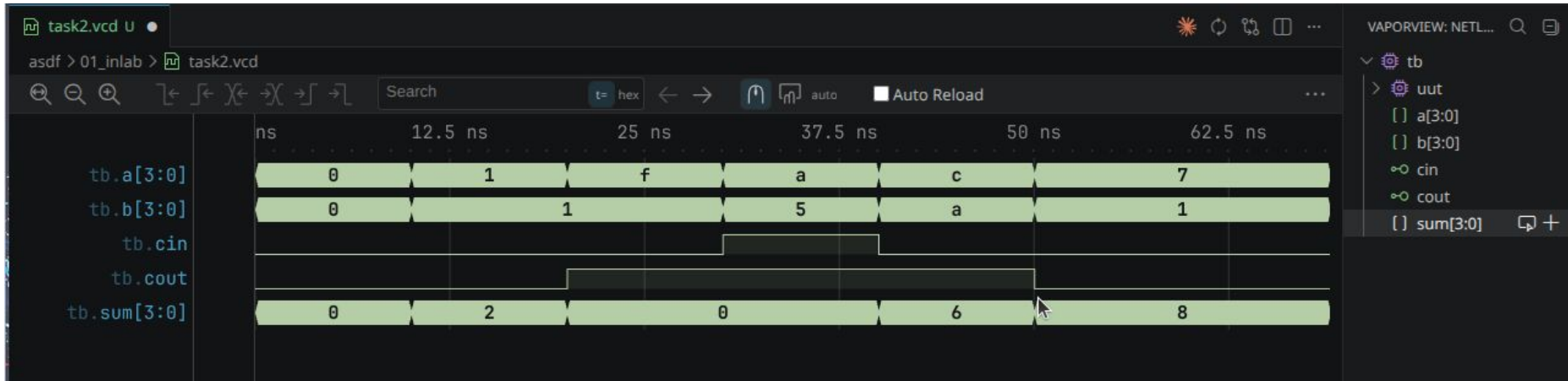
- First, compile **all verilog files** using **iverilog** into a vvp output
- Second, run the simulation using **vvp**; this will execute the blocks in the testbench and **\$display/\$dumpfile+dumpvars** suitable output
- Third, open the (optionally) generated **.vcd** file in **Vaporview** in **VS Code**

Seeing Verilog



- `$dumpfile/$dumpvars` are used to dump signal values over some simulation time to .vcd file
- you can view the waveforms evolving over time in a viewer such as GTKWave, or **Vaporview on VS Code**
- select the signals for display in the **Netlist View**, in the **Vaporview tab in the sidebar**
- click **Zoom to Fit**; then click anywhere (eg. 69ns here) to see values at that point

Seeing Verilog



- vector signals can also be inspected